

Ain Shams University
Faculty of Engineering
Computer Engineering and Software Systems



CFI - Care: Collaborative Flow of Intelligence for Care

Graduation Project Thesis

Author: Mohamed Salah Fathy 21P0117
Youssef Tamer Hossam 21P0138
Salma Hisham Hassan Wagdy 21P0124
Salma Mohamed Abdelmonem 21P0148
Belal Anas Awad 21P0072
Supervisors: Dr Haitham Azmi

Submission Date: 04 April, 2025

Mohamed Salah Fathy 21P0117
Youssef Tamer Hossam 21P0138
Salma Hisham Hassan Wagdy 21P0124
Salma Mohamed Abdelmonem 21P0148
Belal Anas Awad 21P0072
Date: 04 April, 2025

Abstract

Healthcare today is characterized by fragmented experiences where patient medical information is often dispersed across various institutions and providers. This project proposes a comprehensive, patient-centric healthcare application designed to consolidate all medical reports into a single, secure, and unified platform. The system employs Role-Based Access Control (RBAC) to ensure privacy and controlled information sharing. To enhance clinical decision-making, the platform features a medical history visualization graph for tracking health trends and integrates a Retrieval-Augmented Generation (RAG)-enabled AI model to generate diagnostic reports and insights. By transforming isolated data into an interconnected ecosystem, this project aims to establish meaningful relationships between patients, caregivers, and doctors for personalized healthcare delivery.

Contents

1	Introduction	1
1.1	Digital Health Market Overview	3
1.2	HL7 - Health Level Seven	3
1.2.1	Evolution of Standards	3
1.3	Fast Healthcare Interoperability Resources (FHIR)	4
1.3.1	Core Architecture	4
1.3.2	Key Technical Advantages	4
1.3.3	Comparison of Standards	5
2	Literature Review	7
2.1	Authentication and Authorization Standards	7
2.1.1	OAuth 2.0 and OIDC	7
2.1.2	JSON Web Tokens (JWT)	7
2.1.3	PKCE	7
2.2	Healthcare Security and Standards	7
2.2.1	FHIR and SMART on FHIR	7
2.3	Token-Based Authentication Patterns	8
2.3.1	Session Management	8
2.4	API Security	8
2.4.1	Rate Limiting	8
2.4.2	Input Validation and XSS	8
2.4.3	CSRF	8
2.5	Mobile Application Security	8
2.5.1	Secure Token Storage	8
2.6	Audit Logging and Compliance	8
2.6.1	Security Event Logging	8
2.6.2	Threat Modeling	9
2.7	Infrastructure and Deployment	9
2.7.1	API Gateway	9
3	Proposed Design Architecture	10
3.1	Backend	10
3.2	Frontend	10
3.3	Mobile	10

4	Security	11
4.1	Introduction	11
4.2	Architecture & Design	11
4.2.1	Identity & Access Management	13
4.2.2	Authorization & Policy Enforcement	17
4.2.3	Doctor–Patient Consent & Access Grants	17
4.2.4	Session Management & Revocation	18
4.2.5	Defense-in-Depth	20
4.3	Implementation & Setup	21
4.3.1	Keycloak Configuration	21
4.3.2	Backend-for-Frontend (BFF) Security Implementation	22
4.3.3	nginx API Gateway	23
4.3.4	Container Security & Deployment	24
4.3.5	Frontend & Mobile Clients	24
4.3.6	Angular Frontend Security Implementation	27
4.3.7	Keycloak	28
4.3.8	Redis	28
4.3.9	nginx API Gateway	28
4.3.10	PostgreSQL	28
4.3.11	Node.js Backend-for-Frontend (BFF) Libraries	29
4.4	Challenges & Trade-offs	29
4.4.1	Session Management Across Client Types	29
4.4.2	Stateless Tokens vs. Revocation Needs	29
4.4.3	Balancing Consent Privacy & Usability	30
4.4.4	Clock Skew & Distributed Verification	30
4.5	Security Analysis	30
4.5.1	Attack Surface Analysis	30
4.5.2	Threat Model & Residual Risks	31
4.5.3	Mitigations Implemented & Gaps	31
4.6	Summary	32
5	AI Integration	34
6	Testing and Verification	35
7	Conclusion and Future Work	36
	Appendix	37
A	Lists	38
	List of Abbreviations	38
	List of Figures	39
	References	40

Chapter 1

Introduction

Today, patients often face fragmented healthcare experiences, with their medical information dispersed across multiple institutions and providers. This project proposes a patient-centric healthcare application designed to consolidate all medical reports within a single, secure, and unified platform.

The system empowers patients to manage their health through continuous data collection, secure communication, and controlled information sharing using Role-Based Access Control (RBAC) to ensure privacy, consent, and data integrity. Additionally, by integrating features such as medical history visualization and Retrieval-Augmented Generation (RAG)-enabled AI, the proposed system transforms previously scattered and isolated medical data into a unified, interconnected ecosystem. This integration establishes meaningful relationships between patients, healthcare providers, and medical records, promoting continuous, coordinated, and personalized healthcare delivery.

Purpose

The primary purpose of this project is to develop a unified healthcare platform that securely consolidates patients' medical records from multiple institutions. By leveraging technologies such as RAG-enabled AI and graphical data visualization, the project aims to enhance clinical insight and support medical decision-making. Furthermore, the platform seeks to empower patients to monitor their own health data while maintaining full control over information sharing through robust RBAC mechanisms. Ultimately, this initiative strives to ensure privacy, data integrity, and interoperability across the healthcare ecosystem to enhance patient trust and system reliability.

Objective

The key objectives of this project are as follows:

- To develop a unified healthcare platform that securely consolidates patients' medical records from multiple institutions.
- To empower patients to manage and monitor their own health data while maintaining full control over information sharing through RBAC mechanisms.
- To enable secure and efficient communication between patients, doctors, and caregivers.
- To provide data visualization tools, including a medical history graph, to help healthcare providers interpret patient data trends and clinical progress.
- To integrate RAG-enabled AI capabilities to generate automated reports and actionable insights that support doctors in diagnosis and medical decision-making.
- To ensure privacy, data integrity, and interoperability across the healthcare ecosystem to enhance patient trust and system reliability.
- To implement a defense-in-depth security architecture leveraging OAuth 2.0, JWT, and FHIR Consent resources for multi-layered authentication, authorization, and access control across browser and mobile clients.
- To establish comprehensive audit logging, session management, and revocation capabilities for forensic analysis and immediate access termination.

1.1 Digital Health Market Overview

The global push towards digital transformation in healthcare is driven by the need for efficiency, accessibility, and improved patient outcomes. According to Grand View Research, the global digital health market size was valued at USD 288.55 billion in 2024 and is projected to reach USD 946.04 billion by 2030, expanding at a Compound Annual Growth Rate (CAGR) of 22.2% from 2025 to 2030 [1].

This rapid growth is fueled by factors such as the increasing penetration of smartphones, advancements in internet connectivity, and government initiatives promoting digital health infrastructure [1]. However, realizing this potential requires overcoming the challenge of data silos. Currently, healthcare data is often locked within proprietary systems, making *interoperability*—the ability of different information systems to access, exchange, integrate, and cooperatively use data in a coordinated manner—a critical requirement for the modern healthcare ecosystem [3].

1.2 HL7 - Health Level Seven

Health Level Seven (HL7) International is a standards development organization that has defined the framework for the exchange, integration, sharing, and retrieval of electronic health information for decades [2]. Understanding the evolution of these standards is crucial to appreciating the architectural shifts in modern healthcare applications.

1.2.1 Evolution of Standards

- **HL7 v2 (The Workhorse):** Established in 1989, HL7 v2 remains the most widely implemented standard for hospital workflows (e.g., ADT - Admission, Discharge, Transfer messages). It utilizes a non-XML, pipe-delimited syntax (e.g., `MSH|^~\&|...`). While highly efficient for legacy backend systems, its lack of semantic strictness leads to significant implementation variances, posing challenges for modern web and mobile application development [2].
- **HL7 v3 (The Theoretical Model):** Introduced in the 1990s to address the vagueness of v2, HL7 v3 was built on a rigorous Reference Information Model (RIM). It enforced strict semantic correctness using complex XML structures. However, this theoretical purity resulted in a steep learning curve and bloated message sizes, leading to poor adoption rates and a lack of backward compatibility with v2 [2].
- **HL7 CDA (Clinical Document Architecture):** A component of v3, CDA focuses on the exchange of clinical documents (such as discharge summaries). While effective for static snapshots of care, it is document-centric rather than data-centric, making it difficult to extract granular data (like a single heart rate reading) required for real-time interactive applications [4].

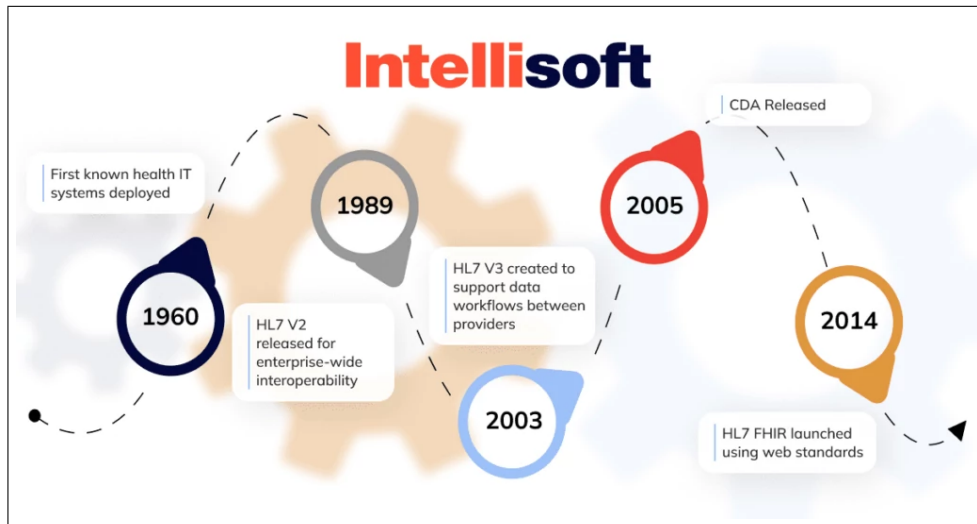


Figure 1.1: Evolution of HL7 Standards from Messaging to RESTful Resources.

1.3 Fast Healthcare Interoperability Resources (FHIR)

In response to the complexity of HL7 v3 and the limitations of v2, HL7 introduced **FHIR** (Fast Healthcare Interoperability Resources) in 2012. FHIR represents a paradigm shift from complex, custom healthcare protocols to modern web standards.

1.3.1 Core Architecture

Unlike its predecessors, FHIR is built on the RESTful API approach widely used in modern IT industries (e.g., by Google, Facebook, Twitter). It treats data elements as independent "Resources"—modular components like **Patient**, **Observation**, or **Medication**—that can be accessed and manipulated via standard HTTP methods (GET, POST, PUT, DELETE) [4].

1.3.2 Key Technical Advantages

- **Granularity via Resources:** FHIR allows systems to access specific data elements using unique URLs (e.g., `GET /Patient/123`), rather than exchanging entire heavy documents. This makes it ideal for mobile devices with limited bandwidth [4].
- **Modern Formats (JSON & XML):** While previous standards relied on complex XML or pipe-delimited text, FHIR natively supports JSON (JavaScript Object Notation), which is lightweight and directly parsable by web browsers and mobile apps [2].

- **The 80/20 Rule:** FHIR focuses on defining the 20% of requirements that satisfy 80% of common healthcare use cases. For the remaining complex or local requirements, it uses a built-in extension mechanism, ensuring the core standard remains simple and implementable [3].

1.3.3 Comparison of Standards

Table 1.1 highlights the fundamental differences between the major HL7 standards, illustrating why FHIR was chosen as the interoperability backbone for this project.

Table 1.1: Comparison of HL7 v2, v3, CDA, and FHIR Standards

Feature	HL7 v2	HL7 v3 / CDA	FHIR
Primary Paradigm	Event-Driven Messaging	Reference Model (RIM) / Documents	RESTful API & Resources
Data Format	Pipe-delimited text	Complex XML	JSON, XML, RDF
Transport	LLP / MLLP (TCP/IP)	SOAP / Web Services	HTTP / REST
Complexity	Moderate (Vague semantics)	High (Steep learning curve)	Low (Web-standard based)
Use Case	Hospital Backend (ADT, Orders)	Clinical Documents (Summaries)	Mobile, Web, Cloud, & Backend

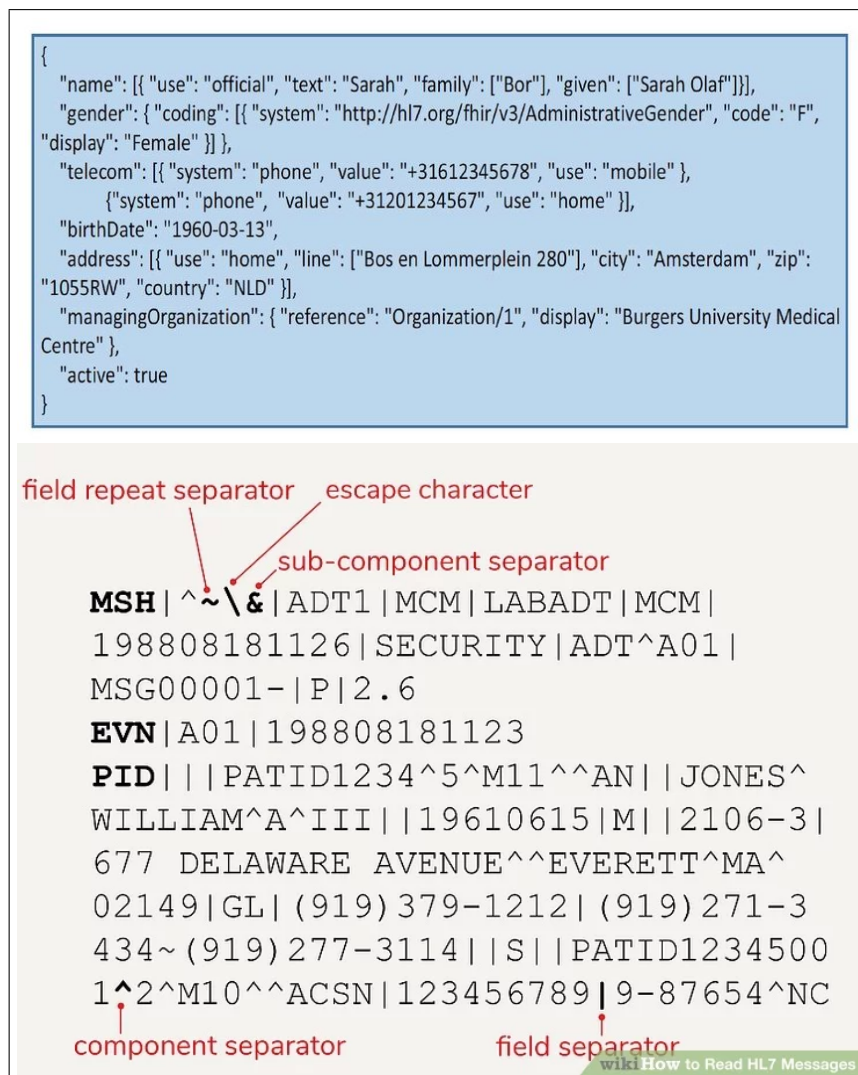


Figure 1.2: Visual Comparison of Data structure: HL7 v2 Message vs. FHIR JSON Resource.

Chapter 2

Literature Review

2.1 Authentication and Authorization Standards

2.1.1 OAuth 2.0 and OIDC

OAuth 2.0 [9] defines authorization grant flows: Authorization Code (web), Resource Owner Password (custom UI), Client Credentials (service-to-service). OpenID Connect (OIDC) [8] adds an identity layer with ID tokens and SSO. CFI-Care uses OAuth 2.0 Resource Owner Password flow with OIDC for token-based API access.

2.1.2 JSON Web Tokens (JWT)

JWTs [11] are signed tokens encoding claims (iss, aud, exp, roles, scopes). Stateless API authentication validates signatures without database queries. Token revocation requires server-side tracking or short expiration windows.

2.1.3 PKCE

PKCE [15] prevents authorization code interception attacks via code verifier/challenge. Recommended for all OAuth 2.0 flows [12].

2.2 Healthcare Security and Standards

2.2.1 FHIR and SMART on FHIR

FHIR [5] defines RESTful APIs for healthcare data exchange. SMART on FHIR [7] applies OAuth 2.0 to FHIR with scoped access (e.g., `patient/Observation.read`). The FHIR consent resource [6] models patient-controlled access with time-bound and actor-specific rules.

2.3 Token-Based Authentication Patterns

2.3.1 Session Management

HTTP-only cookies prevent JavaScript access [14]; SameSite blocks CSRF. token revocation uses redis-backed session tracking or OAuth 2.0 server revocation.

2.4 API Security

2.4.1 Rate Limiting

Rate limiting mitigates brute-force and enumeration attacks via Redis-backed counters [25].

2.4.2 Input Validation and XSS

Schema validation (Joi) enforces format/complexity rules. OWASP [22] recommends output encoding and CSP headers.

2.4.3 CSRF

CSRF [23] is mitigated with SameSite cookies and CSRF tokens.

2.5 Mobile Application Security

2.5.1 Secure Token Storage

Android Keystore [33] provides OS-level encryption and access control, unlike browser localStorage.

2.6 Audit Logging and Compliance

2.6.1 Security Event Logging

OWASP [27] specifies what to log and exclude audit logs enable forensics and compliance.

2.6.2 Threat Modeling

RFC 6819 [16] outlines OAuth 2.0 threats: token theft, interception, phishing, CSRF, replay.

2.7 Infrastructure and Deployment

2.7.1 API Gateway

nginx[24] terminates TLS, routes requests, enforces rate limits, and sets security headers.

Chapter 3

Proposed Design Architecture

3.1 Backend

3.2 Frontend

3.3 Mobile

Chapter 4

Security

4.1 Introduction

Healthcare information systems are prime targets for security breaches due to the commercial value of protected health information (PHI) . CFI-Care platform presents multi-layered challenges: securing web-based browser access, mobile device authentication, sensitive patient consent grants, and doctor-patient access workflows. Also maintaining audit trails is required and so is enabling session revocation.

Traditional monolithic security approaches are insufficient for this use case. Instead, we adopted a defense-in-depth strategy leveraging modern standards (OAuth 2.0/OIDC with JWT) and distributed validation across the API gateway (nginx), backend-for-frontend (Node.js/BFF) and FHIR server for final checks before database access.

4.2 Architecture & Design

The CFI-Care security architecture follows a defense-in-depth model with multiple validation layers. Figure 4.1 provides an overview of the security components and their interactions. Figures 4.2, 4.4, and 4.5 illustrate the key authentication and authorization flows across browser, mobile workflows. This section details the design and implementation principles.

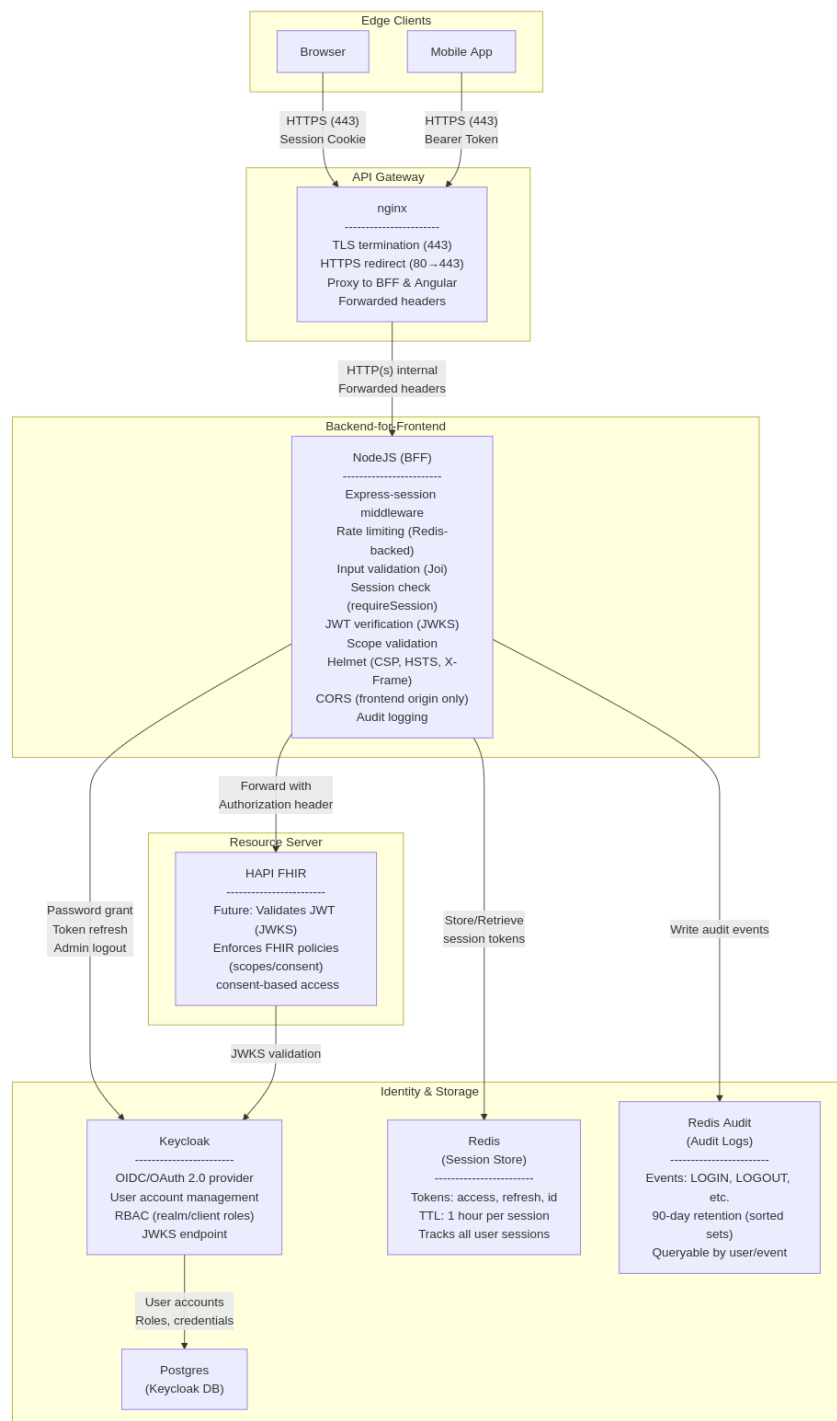


Figure 4.1: Security Architecture Components: Edge clients (Browser, Mobile), API Gateway (nginx), Backend-for-Frontend (Node.js), Identity & Storage (Keycloak, Redis, Postgres), and Resource Server (HAPI FHIR) with their security-focused interactions.

4.2.1 Identity & Access Management

Keycloak Realm & Clients

Keycloak is our chosen identity provider. A dedicated realm hosts a client for the web application (via the BFF) and a client for mobile usage. For the current browser flow, the BFF obtains tokens using the Resource Owner Password grant (password grant) and stores them server-side. Access, refresh, and ID tokens are issued as JWTs and validated downstream using JWKS.

Browser Flow

The browser authenticates against a custom login endpoint exposed by the BFF. On success, the BFF stores tokens in a Redis-backed session and returns a secure, HTTP-only cookie. All subsequent browser requests carry only the session cookie (no tokens in the browser).

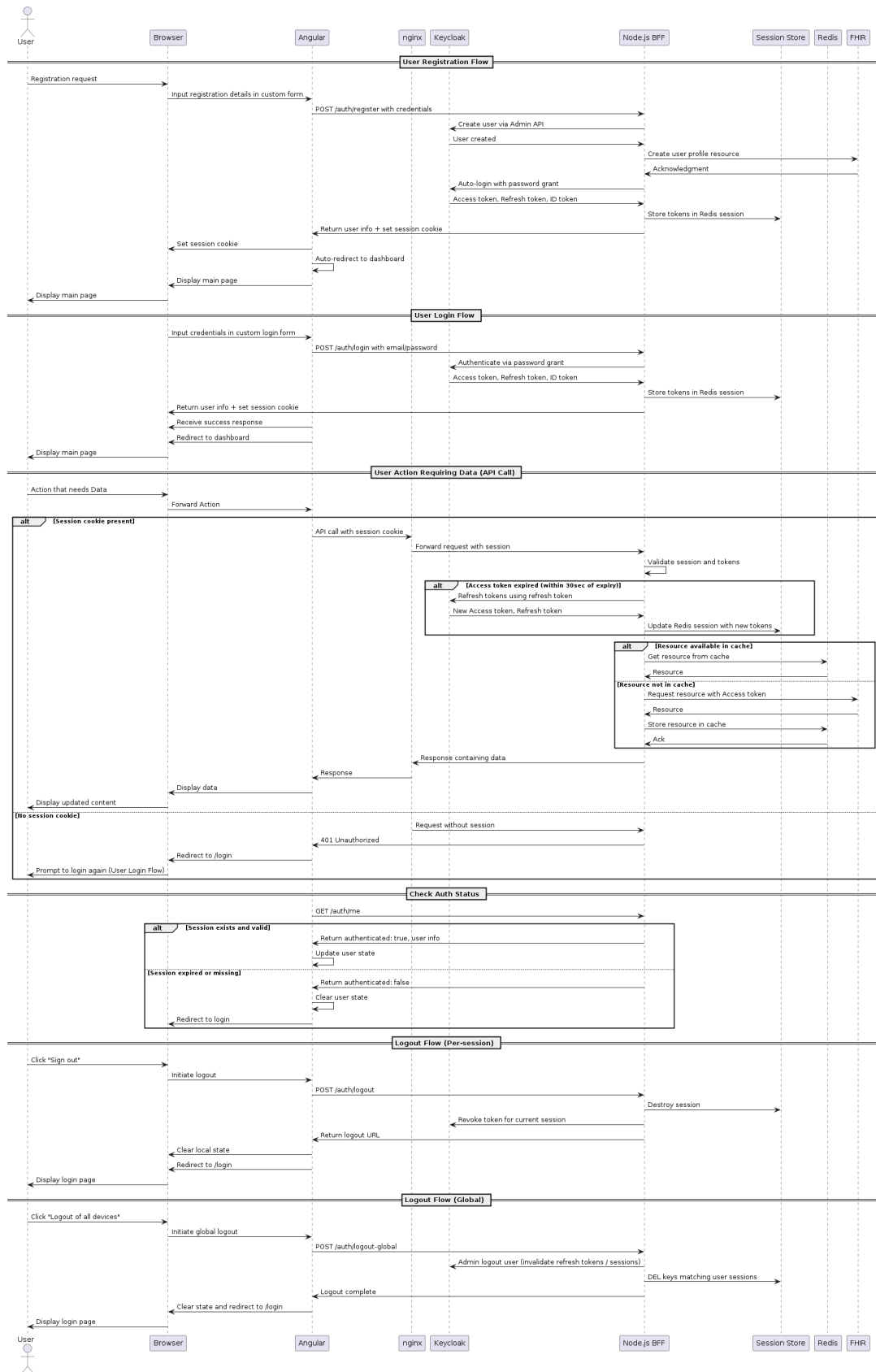


Figure 4.2: Browser Token Authentication Flow: User registration, login, and API request handling with session cookies.

Planned Browser Hardening (Email Verification) A future iteration will keep the custom Angular UI while hardening login with email verification and a MFA option. Tokens remain short-lived, stored in Redis-backed sessions for browsers, and refreshed transparently by the BFF. The planned flow is captured in Figure 4.3.

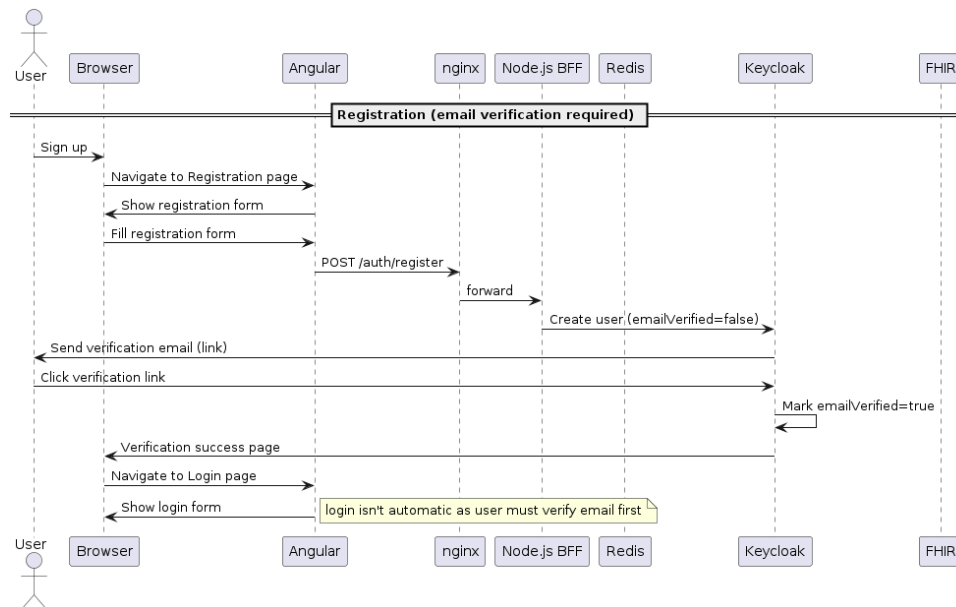


Figure 4.3: Planned Browser Flow (Future): Registration with email verification and BFF-mediated credential login, session cookie issuance, and server-side token refresh.

Mobile Flow

For mobile clients (implemented using Flutter), the flow differs from browsers. The planned implementation uses a BFF-mediated Authorization Code pattern with PKCE, and stateless access tokens. Tokens are stored only in platform-specific secure storage (Android Keystore) and sent as `Authorization: Bearer <token>` in API requests. Token refresh is client-driven via a refresh endpoint, access tokens remain short-lived.

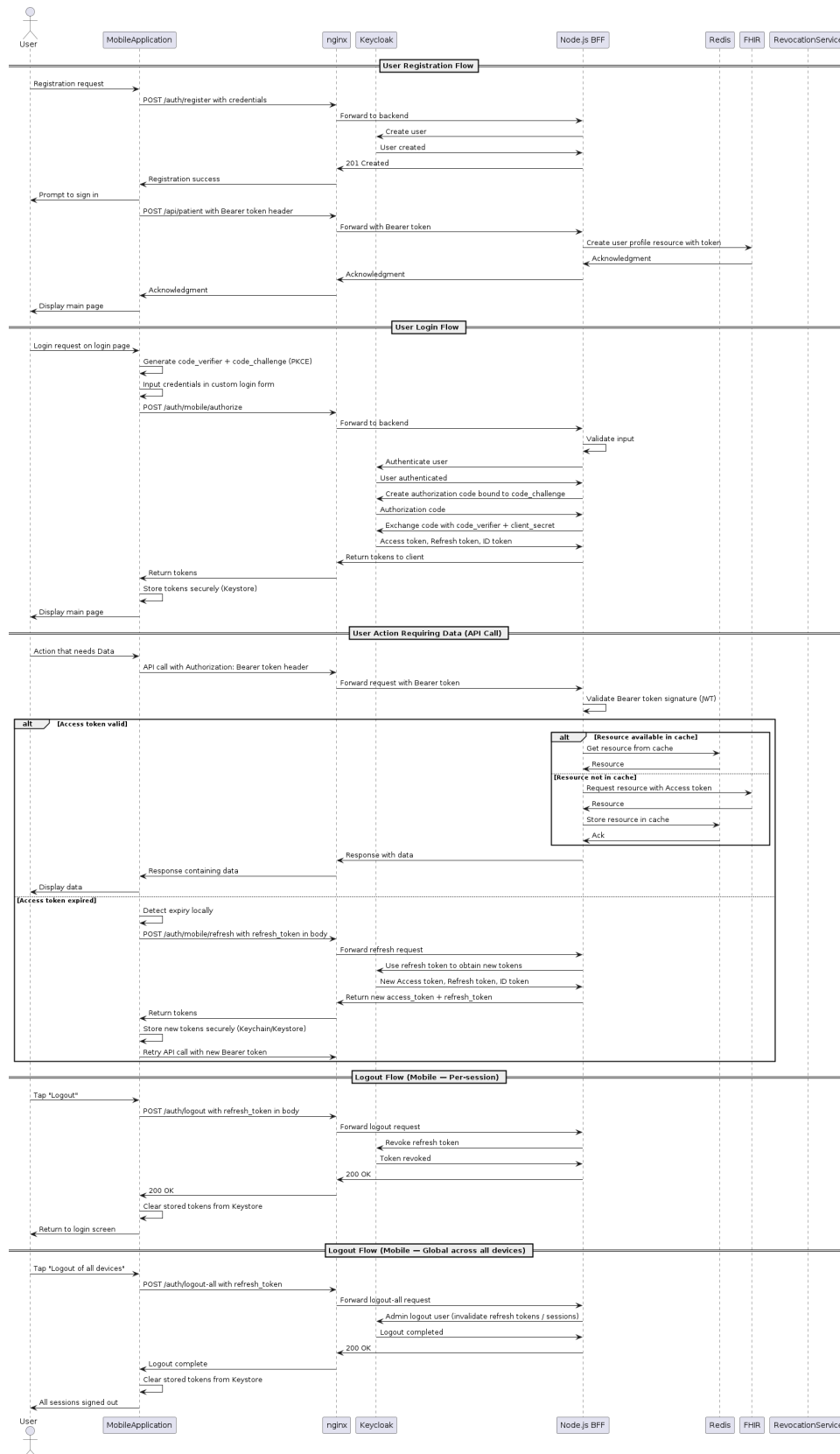


Figure 4.4: Mobile Token Authentication Flow (Future): BFF-mediated Authorization Code with PKCE, secure keystore token storage, and client-driven token refresh.

Session Storage

Browser sessions are maintained using express-session with RedisStore. The BFF saves access/refresh/ID tokens and their expirations. A refresh routine renews tokens when close to expiry. Session cookies are configured for Secure + HttpOnly, and SameSite according to deployment needs.

4.2.2 Authorization & Policy Enforcement

BFF Middleware: Session Check, JWT Verify, Scope Validation

- **Session check (requireSession method):** verifies presence of user and tokens in the Redis-backed session. Automatically refreshes expired tokens by calling Keycloak's token endpoint and persists the updated tokens back to Redis. Populates `req.accessToken`, `req.user`, and `req.kauth` for downstream middleware.
- **JWT verification (verifyToken method):** validates JWT signatures against Keycloak's JWKS endpoint, checks issuer (iss), audience (aud), algorithms, and clock skew tolerance. The Authorization header is populated by `attachForwardedToken` middleware, which extracts the session token from `req.accessToken` (set by `requireSession`) or request headers in general.
- **Scope checks (validateScopes method):** parses `scope` claim and `realm_access.roles` from the token. Compares required SMART on FHIR scopes (e.g., `patient/<id>.rs`) against token scopes using pattern matching. For patient-specific scopes, validates that the `patient` claim in the token matches the requested resource ID.

FHIR Server Enforcement

Future work: validate JWTs independently to implement defense in depth, leverage FHIR server's built-in SMART on FHIR policy enforcement for resource-level access control based on scopes and patient context.

4.2.3 Doctor–Patient Consent & Access Grants

The intended model supports patient-granted access for doctors, with grants stored in Redis (grant key, scopes, TTL) and mirrored as FHIR Consent resources. FHIR Consent resources are created with patient verification (2FA, PIN...) and include provisions specifying scoped access (duration for example). Access grants are cached in Redis for performance but the FHIR Consent is the authoritative source. The BFF verifies an active grant before calling FHIR for doctor requests. Future work: finalize consent persistence, link to FHIR Consent, and enforce patient binding in scope validation.

4.2.4 Session Management & Revocation

The BFF exposes logout endpoints:

- Per-session logout: destroys the current session in Redis and revokes refresh tokens at Keycloak's revoke endpoint.
- Global logout: removes all user sessions and revokes refresh tokens at Keycloak.

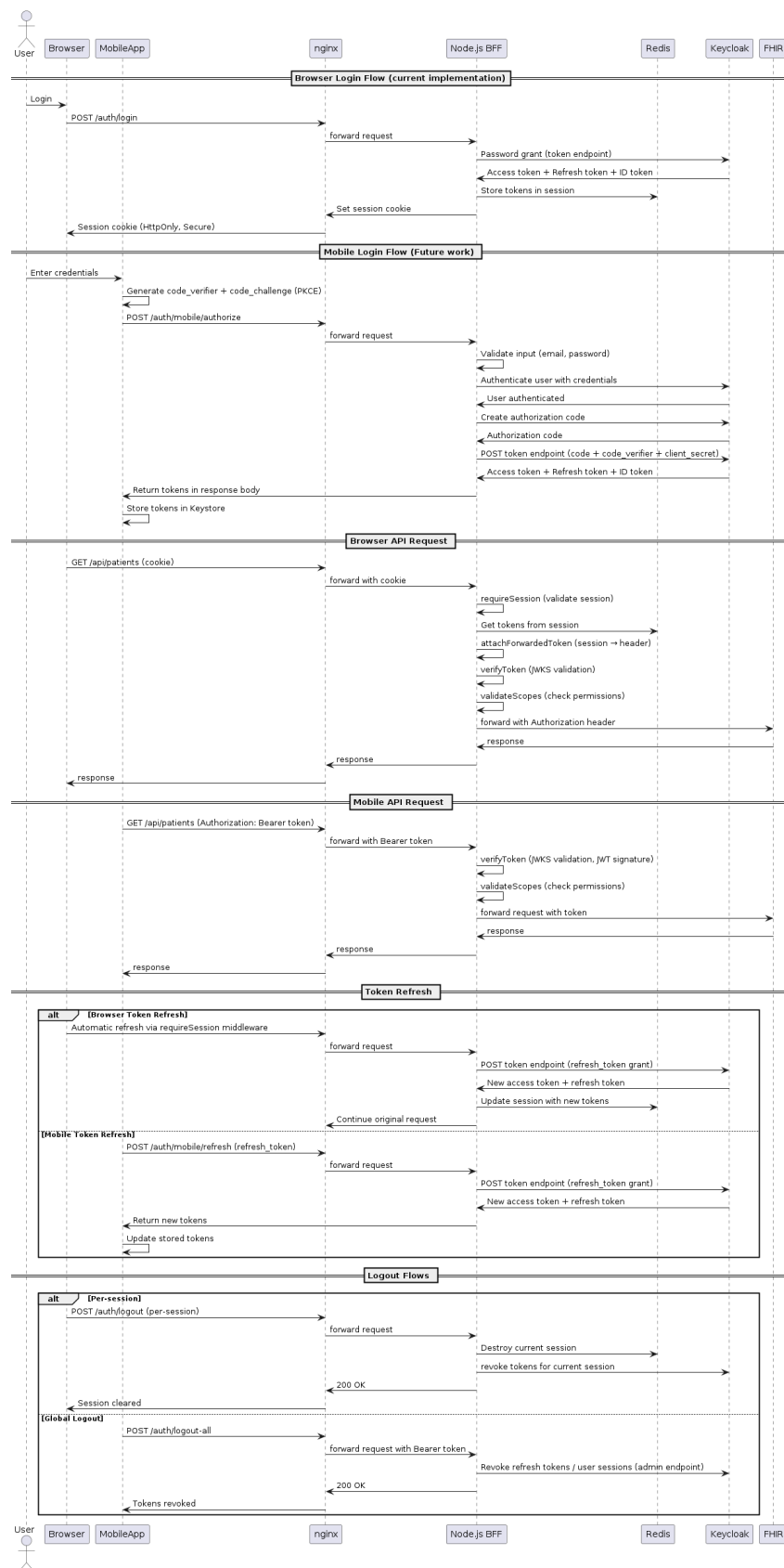


Figure 4.5: Token Flow Overview: Multi-client architecture (browser vs. mobile), session/token management, per-session and global logout patterns.

4.2.5 Defense-in-Depth

Edge

- Browser: session cookies with Secure, HttpOnly, SameSite attributes, route guards, HTTP interceptors for 401 handling.
- Mobile: secure token storage, bearer token auth, client-driven token refresh.

API Gateway (nginx)

- nginx terminates TLS (certs mounted in the container), forces HTTPS (80→443), and proxies `"/api"` and `"/auth"` to the BFF, and `"/"` to the Angular app.
- Forwarded headers are set for correct cookie and session handling.
- Future work: enable HSTS, add common security headers, and tune timeouts.

BFF

Input Validation & Rate Limiting All authentication endpoints (login, register) validate input using Joi schemas before processing. Login requires valid email and password (min 8). Register requires valid email, password with complexity (uppercase, lowercase, digit, min 8 chars), and full name (alphanumeric + spaces). Validation errors are returned as HTTP 400 with field-level details for client feedback.

Rate limiting is implemented via three Redis-backed limiters using `rate-limit-redis` middleware:

- **loginLimiter**: 5 attempts/10 minutes (production) or 100/10 minutes (development so we could keep testing the system).
Counts only failed attempts (`skipSuccessfulRequests: true`).
- **registerLimiter**: 3 attempts/hour (production) or 100/hour (development).
- **generalLimiter**: 100 attempts/10 minutes (production) or 1000/10 minutes (development). Applied globally to all API routes.

Exceeded limits return HTTP 429 (too Many Requests), protecting endpoints from credential enumeration and brute-force attacks.

Security Event Logging & Audit Trail All authentication events (LOGIN_SUCCESS, LOGIN_FAILURE, REGISTER, LOGOUT, LOGOUT_ALL) are logged to Redis sorted sets with timestamps and a 90-day retention period. Each audit log entry captures:

- Event type and timestamp (Unix epoch).
- User ID and email.
- Client IP address and User-Agent header.
- Request ID (UUID) for tracing.
- Metadata.

Audit logs are queryable by event type or by user, enabling security monitoring and forensic analysis.

Session Revocation & Logout The BFF tracks all active sessions for each user in Redis. Per-session logout (`/logout`) destroys the current session and revokes current session's tokens at Keycloak. Global logout (`/logout-all`) clears all stored sessions for the user, rendering all active tokens invalid at the BFF layer. The frontend can then redirect to (`/login`), ensuring re-authentication for subsequent operations.

Resource Server (FHIR Policies)

Future work:

- enforce JWT validation at FHIR server and apply scope and consent-based rules.
- codify patient-specific access constraints for doctor roles.

4.3 Implementation & Setup

4.3.1 Keycloak Configuration

Clients, Scopes

The BFF requests valid scopes. Keycloak is be configured to permit these scopes for the client based on the user's role and map these roles to appropriate SMART scopes.

Password Policy

Registration validates password complexity (length and character classes) in the BFF and matches Keycloak's password policy.

4.3.2 Backend-for-Frontend (BFF) Security Implementation

Session Middleware (Redis), Token Refresh, Token Rotation

Express-session with RedisStore persists session state, including user identity, access token, refresh token, ID token, and their expiration times. A refresh action runs in the `requireSession` middleware: when a token approaches expiration, the BFF automatically sends a refresh token grant to Keycloak's token endpoint, obtains a new access token, and persists the updated values back to Redis. This refresh is transparent to the frontend.

Session cookies are configured with:

- **Secure:** over HTTPS only
- **HttpOnly:** prevents JavaScript access, mitigating XSS token theft.
- **SameSite:** set to `Lax`, blocking CSRF attacks while allowing safe cross-site navigation.
- **MaxAge:** 1 hour so users must re-authenticate on the browser after inactivity.

JWT Verification (JWKS), Scope Checks, Patient Binding

JWKS-based verification validates token signatures using Keycloak's public key endpoint. Checks include:

- Signature validity (RS256 algorithm).
- Issuer claim (`iss`) matches Keycloak's realm URL.
- Audience claim (`aud`) includes the BFF's client ID.

Scope checks compare the token's `scope` claim against required route scopes using pattern-matching logic that handles wildcards. patient-specific scopes require the token's `patient` claim to match the requested resource Id.

Input Validation, Error Handling

All user input (login email, registration password, full name) is validated using Joi schemas before processing. Invalid requests return HTTP 400 with field-level error details, enabling the frontend to display specific guidance. Error responses explicitly do not leak sensitive system details, reducing information disclosure risks.

Mobile Authentication Endpoints (Future Work)

Three dedicated endpoints support mobile client authentication flows:

POST /auth/mobile/login: Authenticates against Keycloak using the Resource Owner Password grant. Upon success, returns `{access_token, refresh_token, id_token, expires_in, user}` in the response body. The mobile app stores these tokens in secure storage (Keystore) and includes the access token in subsequent API requests via **Authorization: Bearer <token>** headers. Rate limiting and input validation (Joi schemas) are enforced. Audit logged as `MOBILE_LOGIN_SUCCESS` or `MOBILE_LOGIN_FAILURE` to separate browser and mobile authentication for auditing granularity.

POST /auth/refresh Accepts `{refresh_token}` in the request body and exchanges it for a new access token via Keycloak's refresh token grant endpoint. Returns `{access_token, refresh_token, expires_in}`. Called client-side by the mobile app when the access token expires. Enables stateless token refresh. Audit logged as `TOKEN_REFRESH` or `TOKEN_REFRESH_FAILURE`.

POST /auth/logout (will be updated for mobile) Dual-path logout supporting both browser sessions and mobile bearer tokens. Mobile path: detects **Authorization: Bearer** header, revokes tokens at Keycloak (via revoke tokens endpoint), and returns `{ok: true}`. Browser path: uses `req.session`, destroys the session, and returns `{ok: true, logoutUrl}`. Requires `refresh_token` in body for Keycloak-side token revocation. Audit will also be logged with client type specified to enable distinction between logout types.

All three endpoints are protected by rate limiters and log to the audit trail for security monitoring.

4.3.3 nginx API Gateway

TLS Termination, HSTS, Secure Headers

TLS is terminated at nginx using valid certificates (self-signed for dev, will be CA-signed for production using Let's encrypt). HTTP traffic on port 80 is redirected to HTTPS (port 443) to ensure all data in transit is encrypted.

The BFF Express server is configured with Helmet, which sets security headers:

- **Content-Security-Policy (CSP):** restricts script sources to `'self'`, reducing XSS attack surface.
- **X-Frame-Options:** prevents clickjacking by restricting iframe embedding.
- **HSTS (Strict-Transport-Security):** forces HTTPS for 1 year, preventing downgrade attacks.

Proxy Configuration, Forwarded Headers

nginx proxies requests to the BFF ("/api" and "/auth" routes) and the Angular frontend ("/" route). The `trust proxy` setting is enabled in Express, allowing it to correctly interpret `X-Forwarded-For`, `X-Forwarded-Proto`, and other headers set by nginx. This ensures:

- Session cookies are correctly marked as Secure (based on proxied HTTPS connection).
- Client IP addresses are correctly logged for audit and rate-limiting purposes.
- Session affinity is maintained across the BFF cluster (when we implement cloud deployment and load balancing).

CORS Configuration

CORS is configured at the BFF level to accept requests only from the frontend origin. Credentials are included in cross-origin requests, allowing the browser to send session cookies. This prevents unauthorized external origins from accessing API endpoint.

4.3.4 Container Security & Deployment

Docker Compose orchestrates five security services: nginx (TLS gateway), Keycloak (identity provider), Postgres (user database), and two Redis instances (session store and audit log store). Security practices include:

- **Volume Mounts:** TLS certificates are mounted as read-only volumes from the host.
- **Network Isolation:** Services communicate over a private Docker network; Redis and Postgres are not exposed to external ports (only accessible from BFF and nginx).
- **Image Security:** we used official base images, most of them scanned for vulnerabilities (CVEs).

Future work: add resource limits per container, enable Docker user namespaces.

4.3.5 Frontend & Mobile Clients

Angular Frontend Architecture

The Angular frontend implements several security layers:

Route Guards (Authentication Check) Protected routes (dashboard, med-graph/:id) are guarded by `authGuard`, a route `canActivate` function that:

- Calls `AuthService.checkAuthStatus()` (a custom function that calls our BFF `auth/me` endpoint) at route activation.
- Verifies the response's `authenticated` flag.
- If authenticated, allows route access, otherwise redirects to `/login`.

The guard injection follows Angular's functional guard pattern (`CanActivateFn`), using dependency injection for `AuthService` and `Router`.

Authentication Interceptor (401 Handling) A global HTTP interceptor (`authInterceptor`) wraps all outbound HTTP requests:

- Catches HTTP responses with status code 401 (Unauthorized).
- Logs a warning message (`console.warn`) for debugging.
- Redirects to `/login`, clearing the current route and forcing re-authentication.
- Re-throws the error after routing, allowing components to handle error states.

This provides automatic session expiry handling: if the backend session expires or tokens are revoked, the first 401 response triggers a redirect, preventing continued access to stale sessions.

Credentials Handling (withCredentials Flag) All HTTP requests to auth endpoints (`/login`, `/register`, `/me`, `/logout`, `/logout-all`) include `withCredentials: true`, instructing Angular's HTTP client to:

- Send session cookies with cross-origin requests.
- Receive Set-Cookie headers from the backend.

This enables secure session management via HTTP-only cookies, preventing token exposure in `typescript`.

Error Handling & User Feedback The `AuthService`'s `handleError` method extracts backend error details:

- For HTTP errors with a `details` array (validation errors from the BFF), displays field-specific messages.
- For generic server errors, displays a fallback message ("Server error").
- Logs errors to the browser console for debugging.

Components using `AuthService` subscribe to the returned `Observable` and display errors to users, improving UX on failed login/registration.

Current User State Management The AuthService maintains a `BehaviorSubject<currentUser>` observable:

- Populated on successful login, registration, or `checkAuthStatus`.
- Cleared on logout or failed authentication.
- Subscribed to by components to display user info (name, email) and show/hide UI elements based on auth state.
- Provides a reactive interface: any component injecting AuthService can subscribe to `currentUser$` and react to state changes.

Third-Party Dependencies Future work: Pin exact versions in `package.json` and regularly audit for vulnerabilities (using `npm audit`).

Mobile App (Flutter) Security Architecture

Although not yet fully implemented, the planned mobile architecture includes:

- **Secure Token Storage:**
 - Android: Keystore system.
- **Bearer Token Auth:** Each HTTP request includes `Authorization: Bearer <access_token>` header.
- **Token Refresh Logic:** Mobile app implements its own refresh routine (unlike browser, where the BFF handles refresh):
 - On 401 response, call the BFF's token refresh endpoint.
 - Retry the original request with the new token.
 - If refresh fails, clear tokens and redirect to login.
- **Certificate Pinning** (future work): pin the BFF's TLS cert in the mobile app to prevent MITM attacks by rogue CAs.

Environment-Specific Configuration

The frontend uses two environment files:

- `environment.ts` (development): `production: false, apiUrl: '/api', authUrl: '/auth'`. Uses relative paths, which resolve to the development server or proxied backend.

- `environment.prod.ts` (production): `production: true, apiUrl: '', authUrl: ''`. URLs are currently empty to be populated at deployment time.

This allows the same compiled app to be deployed across environments by injecting the correct API endpoints at runtime (future work: implement environment variable injection at container startup).

4.3.6 Angular Frontend Security Implementation

Route Protection & Guards

All protected routes (dashboard, med-graph/:id) are declared with `canActivate: [authGuard]`. The `authGuard` executes before route activation and checks authentication status by calling the BFF's GET `/auth/me` endpoint. If the request returns `authenticated: false` or fails with 401, the guard redirects to `/login` and cancels route activation. This prevents unauthorized access to protected pages even if a user manually navigates via URL bar.

HTTP Interceptors & 401 Handling

The global `authInterceptor` catches 401 responses from any API endpoint. Upon 401, the interceptor:

- Logs the unauthorized attempt.
- Immediately navigates to `/login`.
- Re-throws the error, allowing components to also react.

This provides a safety net: even if a route guard passes, if the session expires during navigation, the 401 interceptor ensures the user is redirected to re-authenticate.

Cookie-Based Credentials

All auth endpoints use `withCredentials: true`, ensuring the Angular HTTP client:

- Sends session cookies with every request.
- Accepts Set-Cookie headers from the BFF.

Reactive State Management

The `AuthService` uses a `BehaviorSubject` to maintain observable current user state. Any component can subscribe to `currentUser$` and react to authentication changes.

Error Transparency

Backend validation errors (from /register endpoint) include field-level details in the HTTP response's `error.details` array. The AuthService's `handleError` method extracts and formats these messages, allowing the login component to display specific guidance. This improves UX and helps users follow password complexity requirements.

4.3.7 Keycloak

Keycloak serves as the OpenID Connect identity provider. It manages user accounts, issues JWT tokens (access, refresh, and ID tokens), maintains public JWKS endpoints for token verification, and supports role-based access control via realm roles and client roles. In the current setup, the BFF uses the Resource Owner Password grant to obtain tokens on behalf of users.

4.3.8 Redis

Two Redis instances are deployed:

- **Session Store:** backing store for express-session. Persists user sessions, access tokens, refresh tokens, and their expiration times with a 1-hour TTL.
- **Audit Store:** persists authentication events (LOGIN_SUCCESS, LOGIN_FAILURE, REGISTER, LOGOUT, LOGOUT_ALL) as sorted sets. Each entry includes timestamp, user ID, IP, User-Agent, and request ID. Entries expire after 90 days.

Redis also stores user session tracking sets (all session IDs for each user) to enable logout-all operations.

4.3.9 nginx API Gateway

Acts as the TLS termination point, HTTP-to-HTTPS redirect, and reverse proxy. Sets forwarded headers for correct handling of session cookies and client IP logging in the BFF.

4.3.10 PostgreSQL

Keycloak's backing database, storing user accounts, roles, and credentials.

4.3.11 Node.js Backend-for-Frontend (BFF) Libraries

- **express-session and connect-redis**: Server-side session management with Redis persistence, enabling stateful auth for browsers.
- **joi**: Schema validation library for login and registration input (email format, password complexity, full name pattern).
- **rate-limit-redis**: Rate limiting middleware backed by Redis. Protects endpoints from brute-force attacks.
- **jsonwebtoken**: JWT creation and verification library (for validation via JWKS).
- **jwtks-rsa**: Fetches and caches Keycloak's public JWKS endpoint. Automatically refreshes keys on signature validation failure.
- **axios**: HTTP client for outbound calls to Keycloak API.
- **helmet**: Express middleware that sets security headers
- **cors**: Middleware for Cross-Origin Resource Sharing, configured to accept requests from the Angular frontend origin only.

4.4 Challenges & Trade-offs

4.4.1 Session Management Across Client Types

Browsers use server-side sessions with secure cookies, mobile apps use bearer tokens in the Authorization header. Keeping behavior consistent requires:

- Route guards and 401 interceptors on both client types.
- Unified token refresh logic (automatic for browser via `requireSession`, manual for mobile via OAuth refresh endpoint).
- Synchronized revocation: per-session logout on the BFF invalidates tokens server-side, mobile clients must also clear cached tokens locally.

4.4.2 Stateless Tokens vs. Revocation Needs

Access tokens are stateless (signed by Keycloak, verified locally), making them scalable. However, true instant revocation (logout-all) for browsers requires server-side state (Redis session tracking). The current design trades off stateless simplicity for revocation correctness: the BFF maintains session state and validates session presence on every request, also for bearer token users in future deployments.

4.4.3 Balancing Consent Privacy & Usability

Patient-granted access for doctors (consent workflow) must be enforced transparently. Overly strict consent checks may block legitimate clinical workflows, overly permissive checks may expose PHI. The current design is for grants to be stored in Redis (grant key, scopes, TTL) and validated at the BFF, and the FHIR server will apply additional constraints based on consent resources.

4.4.4 Clock Skew & Distributed Verification

JWT validation includes clock skew tolerance (10 seconds) to account for distributed clock drift between Keycloak and the BFF. However, if system clocks drift beyond this window or if Keycloak's keys rotate, validation failures may spike.

4.5 Security Analysis

4.5.1 Attack Surface Analysis

- **Token Theft:** mitigated by storing tokens server-side' not in browser memory. HTTP-only cookies prevent JavaScript access via XSS. TLS prevents interception in transit.
- **Cross-Site Request Forgery:** Session cookies use SameSite=Lax, blocking cross-site submissions. POST endpoints on the BFF do not rely solely on cookies.
- **Credential Enumeration / Brute-Force:** Rate limiters make password guessing infeasible within the policy window.
- **Information Disclosure:** Error messages do not leak system internals, where registration validation errors are field-level and generic. Audit logs are server-side and not exposed to unauthenticated clients.
- **Privilege Level:** Token claims are issued by Keycloak and validated server-side. The BFF doesn't allow clients to modify claims.
- **Patient Privacy Breach:** Scope checks enforce patient binding so doctors cannot access patients without appropriate scopes/consents. FHIR server applies additional resource-level checks.

4.5.2 Threat Model & Residual Risks

Residual risks include:

- **Keycloak Compromise:** If Keycloak is breached, all issued tokens become untrustworthy. Mitigated by: securing Keycloak (strong admin passwords, HTTPS only), and monitoring for unusual token patterns.
- **Denial of Service (DoS):** Backend could be overwhelmed by excessive requests. Mitigated by: rate limiters on API endpoints.
- **XSS Attacks:** While tokens are not in browser memory, XSS could still hijack sessions via cookies. Mitigated by: CSP headers, HttpOnly cookies
- **Credential Leakage:** Users reusing weak passwords across sites risk credential stuffing attacks. Mitigated by: strong password policies, rate limiting, and future MFA implementation.
- **Keycloak's Storage of Credentials:** If Keycloak's database is compromised, user credentials may be exposed. Mitigated by: securing Postgres access, Future work: encryption at rest.
- **Redis Compromise:** Session and audit stores in Redis are unencrypted at rest. Mitigated by: network isolation and future work on Redis encryption.
- **Scope/Role Misconfiguration:** Overly broad scopes or clinician roles may grant unintended access. Mitigated by: audit logging, Future work: more granular access control through keycloak authorization functionalities.
- **Consent Enforcement Gaps:** Current consent model is incomplete, doctor-patient consent is not fully enforced at the FHIR server. Mitigated by: future work on FHIR Consent resources and server-side validation.

4.5.3 Mitigations Implemented & Gaps

Implemented Mitigations

- TLS encryption for all data in transit.
- Session cookies with HttpOnly, Secure, and SameSite flags.
- JWKS-based JWT verification with issuer, audience, and algorithm checks.
- Rate limiting on authentication endpoints (adaptive for dev/prod).
- Input validation for authentication data.
- Audit logging.
- User session tracking for per-session and global logout.
- Security headers via Helmet.

Gaps & Future Work

- **PKCE (Proof Key for Code Exchange):** Current flow uses Resource Owner Password grant, future work will migrate to Authorization Code + PKCE for improved security.
- **Consent Enforcement:** Patient-granted doctor access is not yet implemented or persisted as FHIR Consent resources or validated at the FHIR server.
- **Redis Encryption:** Session and audit data in Redis are unencrypted at rest, future work includes enabling Redis persistence encryption.
- **Centralized Audit Dashboard:** Audit logs are stored in Redis, future work includes creating a dedicated UI for viewing and filtering logs as that would improve security monitoring (low priority future work).
- **Scope / Role Mapping:** Keycloak role-to-SMART-scope mapping needs formalization and documentation.
- **Additional Headers:** CSP and HSTS are implemented; future work includes adding X-Content-Type-Options, Referrer-Policy, and Permissions-Policy.

4.6 Summary

CFI-Care implements a defense-in-depth security architecture across three layers:

Identity & Access (Keycloak + Tokens) OAuth 2.0 with the Resource Owner Password grant issues JWTs signed by Keycloak. Tokens include user identity, scopes, roles, and claims. Keycloak serves as the trusted identity provider, issuing and validating credentials.

Browser Access (Sessions + BFF) The BFF stores tokens server-side in Redis-backed sessions, exposing only secure, HTTP-only cookies to the browser. A per-request token refresh routine ensures tokens never expire, transparently re-authenticating users. This design protects tokens from XSS theft and simplifies client-side logic.

API Layer (Middleware + Validation) Every API request flows through multiple middleware checks: session validation, JWT signature verification, and scope/role enforcement. Rate limiters protect endpoints from brute-force attacks. Input validation prevents injection attacks. Audit logs capture all authentication events for forensic analysis.

Transport & Edge (TLS + Headers) nginx terminates TLS, enforces HTTPS-only access, and sets forwarded headers. The BFF applies Helmet security headers (CSP, HSTS, X-Frame-Options)

Current State The implementation is functional and ready for incremental hardening:

- All authentication endpoints are secured with rate limiting and input validation.
- Token refresh is automatic and transparent to browsers.
- Per-session and global logout are fully functional.
- Audit trails are captured and retained for 90 days.
- Mobile support is architected for bearer token auth (future implementation).

Next Steps

- Implement mobile client authentication endpoints.
- Deploy secrets management using a vault solution
- Deploy security containers to cloud infrastructure with load balancing.
- Use Let's Encrypt for production level TLS certificates after cloud deployment.
- Complete the patient-doctor consent workflow and enforce it at the FHIR server.
- Finalize role-to-scope mapping in Keycloak and document SMART on FHIR compliance.
- Migrate to Authorization Code + PKCE for improved security and standards compliance.
- Build a centralized audit dashboard for security monitoring.
- Enable Redis persistence encryption for session and audit data at rest.

Chapter 5

AI Integration

Chapter 6

Testing and Verification

Chapter 7

Conclusion and Future Work

Conclusion and future work go here.

Appendix

Appendix A

Lists

List of Figures

1.1	Evolution of HL7 Standards from Messaging to RESTful Resources. . . .	4
1.2	Visual Comparison of Data structure: HL7 v2 Message vs. FHIR JSON Resource.	6
4.1	Security Architecture Components: Edge clients (Browser, Mobile), API Gateway (nginx), Backend-for-Frontend (Node.js), Identity & Storage (Keycloak, Redis, Postgres), and Resource Server (HAPI FHIR) with their security-focused interactions.	12
4.2	Browser Token Authentication Flow: User registration, login, and API request handling with session cookies.	14
4.3	Planned Browser Flow (Future): Registration with email verification and BFF-mediated credential login, session cookie issuance, and server-side token refresh.	15
4.4	Mobile Token Authentication Flow (Future): BFF-mediated Authorization Code with PKCE, secure keystore token storage, and client-driven token refresh.	16
4.5	Token Flow Overview: Multi-client architecture (browser vs. mobile), session/token management, per-session and global logout patterns.	19

References

- [1] Grand View Research, *Digital Health Market Size, Share & Trends Analysis Report By Technology, By Component, By Region, And Segment Forecasts, 2025 - 2030*, <https://www.grandviewresearch.com/industry-analysis/digital-health-market>, Accessed: 2025-04-04.
- [2] Intellisoft, *FHIR vs. HL7: Core Differences & Standards Comparison*, <https://intellisoft.io/comparing-hl7-vs-fhir-core-differences/>, Accessed: 2025-04-04.
- [3] University of Porto Repository, *Interoperability in Healthcare Systems (Thesis/Dissertation)*, <https://repositorio-aberto.up.pt/bitstream/10216/164181/2/700384.pdf>, Accessed: 2025-04-04.
- [4] Wikipedia, *Fast Healthcare Interoperability Resources*, https://en.wikipedia.org/wiki/Fast_Healthcare_Interoperability_Resources, Accessed: 2025-04-04.
- [5] HL7 International, *FHIR HL7 Documentation*, <https://www.hl7.org/fhir/>, Accessed: 2025-04-04.
- [6] HL7 International, *FHIR R4 Consent Resource*, <https://www.hl7.org/fhir/consent.html>, Accessed: 2025-11-04.
- [7] HL7 SMART on FHIR Community, *SMART App Launch Framework: Secure, Standardized, Data-Driven Application Architecture for Biomedicine*, <http://www.hl7.org/fhir/smart-app-launch/>, Accessed: 2025-09-15.
- [8] OpenID Foundation, *OpenID Connect Core 1.0*, https://openid.net/specs/openid-connect-core-1_0.html, Accessed: 2025-08-28.
- [9] D. Hardt (Editor), *The OAuth 2.0 Authorization Framework*, RFC 6749, Internet Engineering Task Force (IETF), October 2012, <https://tools.ietf.org/html/rfc6749>, Accessed: 2025-08-28.
- [10] B. Jones and D. Hardt (Editors), *The OAuth 2.0 Bearer Token Usage*, RFC 6750, Internet Engineering Task Force (IETF), October 2012, <https://tools.ietf.org/html/rfc6750>, Accessed: 2025-01-15.

- [11] M. Jones, J. Bradley, and N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), May 2015, <https://tools.ietf.org/html/rfc7519>, Accessed: 2025-08-28.
- [12] O. Steele and A. Parecki (Editors), *OAuth 2.0 Security Best Current Practice*, RFC 9700, Internet Engineering Task Force (IETF), October 2024, <https://tools.ietf.org/html/rfc9700>, Accessed: 2025-11-04.
- [13] M. Bertocci, *OAuth 2.0 Token Mediator Pattern for Web API Authorization*, Internet-Draft draft-bertocci-oauth2-tmi-bff-01, IETF, 2023, <https://tools.ietf.org/html/draft-bertocci-oauth2-tmi-bff-01>, Accessed: 2025-01-14.
- [14] S. Bingle, Ed. *HTTP State Management Mechanism*, RFC 6265, Internet Engineering Task Force (IETF), April 2011, <https://tools.ietf.org/html/rfc6265>, Accessed: 2025-08-30.
- [15] N. Sakimura, J. Bradley, and N. Agarwal, *Proof Key for Public OAuth 2.0 Authorization Code Flow (PKCE)*, RFC 7636, Internet Engineering Task Force (IETF), September 2015, <https://tools.ietf.org/html/rfc7636>, Accessed: 2025-09-30.
- [16] D. Recordon (Editor) and D. Hardt (Editor), *OAuth 2.0 Threat Model and Security Considerations*, RFC 6819, Internet Engineering Task Force (IETF), January 2013, <https://tools.ietf.org/html/rfc6819>, Accessed: 2025-11-04.
- [17] OWASP Foundation, *OWASP Top 10 - 2021: The Ten Most Critical Web Application Security Risks*, <https://owasp.org/Top10/2021/>, Accessed: 2025-09-30.
- [18] OWASP Foundation, *OWASP API Security Top 10*, <https://owasp.org/www-project-api-security/>, Accessed: 2025-11-02.
- [19] Keycloak Community, *Keycloak Documentation: Open Source Identity and Access Management*, <https://www.keycloak.org/documentation.html>, Accessed: 2025-08-20.
- [20] Keycloak, *Keycloak Server Administration Guide and Reference*, https://www.keycloak.org/docs/latest/server_admin/, Accessed: 2025-08-20.
- [21] Adam Baldwin and Contributors, *Helmet: Secure Express Apps by Setting Various HTTP Headers*, <https://helmetjs.github.io/>, Accessed: 2025-01-15.
- [22] OWASP Foundation, *Cross-Site Scripting (XSS) Prevention Cheat Sheet*, https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html, Accessed: 2025-09-30.
- [23] OWASP Foundation, *Cross-Site Request Forgery (CSRF) Prevention Cheat Sheet*, https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html, Accessed: 2025-09-29.

- [24] NGINX Inc., *NGINX Documentation*, <https://nginx.org/en/docs/>, Accessed: 2025-10-04.
- [25] Olivier Poitrey, *rate-limit-redis: Redis-backed Rate Limiting Middleware for Express*, <https://www.npmjs.com/package/rate-limit-redis>, Accessed: 2025-01-15.
- [26] hapi Framework Community, *Joi: Object Schema Validation*, <https://joi.dev/>, Accessed: 2025-01-15.
- [27] OWASP Foundation, *Logging Cheat Sheet*, https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html, Accessed: 2025-10-04.
- [28] U.S. Department of Health and Human Services, *HIPAA Security Rule*, <https://www.hhs.gov/hipaa/for-professionals/security/index.html>, Accessed: 2025-08-23.
- [29] Art Future, *OAuth 2.0 Authentication with Keycloak in Flutter Applications*, <https://medium.com/@art.futurx/oauth-2-0-authentication-with-keycloak-in-flutter-applications-35d588b2246d>, Accessed: 2025-12-04.
- [30] D. Denniss, C. Dodson, and W. Mills, *OAuth 2.0 for Native Apps*, RFC 8252, Internet Engineering Task Force (IETF), October 2017, <https://www.rfc-editor.org/rfc/rfc8252#section-8.12>, Accessed: 2025-12-04.
- [31] OWASP Foundation, *OWASP Mobile Top 10*, <https://owasp.org/www-project-mobile-top-10/>, Accessed: 2025-08-23.
- [32] OWASP Foundation, *OWASP Mobile Application Security (MAS)*, <https://mas.owasp.org/>, Accessed: 2025-08-23.
<https://developer.android.com/privacy-and-security/keystore>
- [33] Google, *Android Keystore System*, <https://developer.android.com/privacy-and-security/keystore>, Accessed: 2025-01-14.
- [34] Auth0, *The Backend-for-Frontend Pattern*, <https://auth0.com/blog/the-backend-for-frontend-pattern-bff/>, Accessed: 2025-12-04.
- [35] Sam Newman, *Backend-for-Frontend Architecture Pattern*, <https://samnewman.io/patterns/architectural/bff/>, Accessed: 2025-12-04.
- [36] Keycloak, *Keycloak Securing Applications and Services Guide*, <https://www.keycloak.org/guides.html#securing-apps>, Accessed: 2025-09-10.
- [37] Keycloak, *Keycloak REST API Documentation*, <https://www.keycloak.org/docs-api/latest/rest-api/index.html>, Accessed: 2025-09-10.

- [38] Keycloak, *Keycloak Authorization Services Guide*, https://www.keycloak.org/docs/latest/authorization_services/index.html, Accessed: 2025-09-10.
- [39] Keycloak, *Keycloak Server in Containers*, <https://www.keycloak.org/server/containers>, Accessed: 2025-09-10.
- [40] Keycloak Documentation, *Administering Sessions in Keycloak*, https://wjlw465150.gitbooks.io/keycloak-documentation/content/server_admin/topics/sessions/administering.html, Accessed: 2025-01-14.